



ACTIVIDAD 2 Y 3

S-SDLC, DEVSECOPS Y EVALUACIÓN DE SEGURIDAD



Metodologías de Desarrollo Seguro - Web-App To-Do

Nombre	Matrícula / usuario UEM
Iván Jiménez Caidas	224c8130
Sergio González Martín	2181960
Daniel Asensio Vázquez	224b9614
Mario Martín Fernández	224a9917

Documento grupal que unifica la Actividad 2 y la Actividad 3 sobre la Web-App To-Do.

5 DE JUNIO DE 2026

ÍNDICE

1. Introducción general	3
2. Actividad 2: incorporación de S-SDLC y DevSecOps	3
2.1. Miembros del grupo y repositorios base	3
2.2. Definición de la aplicación y ejecución	3
Arquitectura prevista.....	3
Comandos de ejecución	4
2.3. Revisión de proyectos individuales	4
2.4. Unificación del proyecto	4
2.5. Consideraciones de seguridad en el diseño	5
2.6. Implementación DevSecOps paso a paso	5
Cultura de seguridad	5
Control de versiones y ramas.....	5
Incorporar seguridad en el desarrollo.....	6
Automatización CI/CD	6
Herramientas de seguridad seleccionadas	6
Flujo de pipeline propuesto	6
Concienciación y mejora continua	7
2.7. Diagrama S-SDLC y DevSecOps	7
3. Actividad 3: evaluación de seguridad mediante herramientas automáticas	8
3.1. Herramientas seleccionadas y justificación	8
3.2. Alteraciones introducidas para generar alertas	8
3.3. Resultados de las pruebas automáticas	10
GitHub Secret Scanning	11
GitHub Dependabot - SCA	11
Snyk - SAST	12
Nmap - escaneo de red	12
OWASP ZAP - DAST	13

3.4. Tabla resumen de resultados.....	13
3.5. Relación con S-SDLC y DevSecOps	14
4. Conclusión conjunta	14

1. Introducción general

Este documento recoge de forma integrada el trabajo desarrollado sobre la **Web-App To-Do** en las actividades de **S-SDLC**, **DevSecOps** y evaluación automática de seguridad. La primera parte mantiene la continuidad de la Actividad 2, donde se definió una versión común del proyecto y se describió cómo incorporar seguridad desde el diseño. La segunda parte amplía ese trabajo con la ejecución de **herramientas automáticas** de análisis de seguridad, tal y como se solicita en la actividad posterior.

El enfoque seguido es práctico: primero se define el proyecto tipo, su arquitectura y su flujo DevSecOps; después se introducen **vulnerabilidades controladas** en la aplicación y en sus ficheros de configuración para comprobar si las herramientas seleccionadas son capaces de generar alertas útiles. Estas modificaciones se realizan únicamente en un **entorno de pruebas** y con finalidad académica.

La idea principal es evidenciar que la seguridad no se limita a revisar el código al final, sino que debe integrarse desde la **planificación, diseño, desarrollo, pruebas, verificación, despliegue y mantenimiento** de la aplicación.

2. Actividad 2: incorporación de S-SDLC y DevSecOps

En esta primera parte se documenta la base del proyecto grupal. Todos los miembros trabajaron sobre una aplicación To-Do, por lo que la unificación no consiste en mezclar aplicaciones distintas, sino en construir una versión común que incorpore las mejores prácticas de seguridad, documentación y automatización propuestas por el grupo.

2.1. Miembros del grupo y repositorios base

La siguiente tabla recoge los integrantes del grupo, su matrícula o usuario UEM y el repositorio utilizado como referencia en la actividad anterior. Se incluyen únicamente los identificadores académicos, evitando correos completos.

Miembro	Matrícula / usuario UEM	Repositorio actividad anterior	Aplicación
Iván Jiménez Caidas	224c8130	Entregado en local	To-Do App
Sergio González Martín	2181960	https://github.com/porteroFitness/Met_Desarrollo_Seguro-To_Do-Secured_A1	To-Do App
Daniel Asensio Vázquez	224b9614	https://github.com/dasensio03/secure-todo-sdlc	To-Do App
Mario Martín Fernández	224a9917	https://github.com/mario2448/Actividad-1-ToDo-Mario	To-Do App

2.2. Definición de la aplicación y ejecución

El proyecto final se basa en una **Web-App tipo To-Do**, utilizada por los miembros del grupo en la actividad anterior. La aplicación permite gestionar tareas desde una interfaz web y se ejecuta mediante **Docker**, lo que facilita un entorno reproducible y aislado para los distintos servicios.

Arquitectura prevista

- Proxy inverso: Traefik.

- Frontend: React.js.
- Backend: Node.js.
- Base de datos: MySQL.
- Gestión de base de datos: phpMyAdmin.
- Contenerización: Docker y Docker Compose.

Comandos de ejecución

Desde la raíz del repositorio se propone ejecutar la aplicación con los siguientes comandos:

```
git clone https://github.com/porteroFitness/Met_Desarrollo_seguro-ToDo-DevSecOps
cd Met_Desarrollo_seguro-ToDo-DevSecOps
cp .env.example .env
docker compose up -d --build
docker ps
```

Para detener los contenedores del entorno se utiliza:

```
docker compose down
```

2.3. Revisión de proyectos individuales

Como todos los miembros trabajaron sobre una aplicación To-Do, el análisis se centra en comparar las medidas de seguridad, documentación y controles propuestos por cada uno. Esta revisión permite construir una versión común más sólida, integrando las aportaciones más útiles de cada repositorio individual.

Miembro	Medidas identificadas	Fase S-SDLC relacionada	Estado
Iván Jiménez Caidas	Enfoque Shift-Left y Business Recovery Plan. Análisis SAST en pull requests. Uso de SCA para generar un SBOM de dependencias. Pruebas DAST y protección activa.	Diseño, implementación, construcción, pruebas, despliegue y mantenimiento.	Aprovechable
Sergio González Martín	Uso de ramas, control de versiones, .env, .gitignore, Trivy, SAST/SCA, Quality Gates y entorno staging.	Diseño, desarrollo, pruebas, verificación y despliegue.	Aprovechable
Daniel Asensio Vázquez	Ejecución Docker, documentación S-SDLC, pruebas con npm audit, estructura docs/security/tests y evidencias.	Requisitos, diseño, implementación, pruebas y mantenimiento.	Aprovechable
Mario Martín Fernández	Usuario no root en Dockerfile, variables de entorno y uso de .gitignore/.dockerignore. Exposición mínima de red, revisión con npm audit y plan de pruebas con Trivy y OWASP ZAP (DAST).	Requisitos, diseño, desarrollo, pruebas y mantenimiento.	Aprovechable

2.4. Unificación del proyecto

La versión común de la To-Do integra las mejores prácticas observadas en los proyectos individuales. El objetivo es mantener una **Web-App** sencilla y funcional, pero con una estructura de seguridad más completa y alineada con **S-SDLC** y **DevSecOps**.

- Ejecución mediante Docker Compose.
- Uso de .env para variables sensibles e inclusión de .env en .gitignore.
- Archivo .env.example con valores de ejemplo para no exponer credenciales reales.
- Revisión de Dockerfile y compose.yaml.
- Uso de ramas main, preprod y feature/fix.

- Revisión de dependencias mediante GitHub Dependabot como SCA.
- Análisis estático SAST con Snyk.
- Detección de secretos con GitHub Secret Scanning.
- Escaneo de imágenes Docker con Trivy.
- Revisión de puertos y servicios expuestos con Nmap.
- Análisis dinámico DAST con OWASP ZAP.
- Quality Gates para bloquear vulnerabilidades altas o críticas.
- Despliegue en staging antes de fusionar a main.

2.5. Consideraciones de seguridad en el diseño

Para el diseño de seguridad se parte de una premisa realista: todas las organizaciones pueden sufrir ataques. El objetivo no es bloquear la productividad, sino reducir el impacto de un incidente y permitir una recuperación adecuada del negocio.

Se aplica el enfoque **Shift-Left**, desplazando la seguridad hacia las fases iniciales del ciclo de vida. Esto permite definir requisitos de seguridad, revisar configuraciones, analizar riesgos y automatizar pruebas antes del despliegue.

Riesgo	Ubicación	Medida propuesta
Credenciales hardcodeadas	compose.yaml	Usar .env y subir solo .env.example.
Exposición accidental de secretos	GitHub	.gitignore y GitHub Secret Scanning.
Uso de versiones latest	phpMyAdmin / imágenes Docker	Fijar versiones estables.
Ejecución como root	Dockerfile	Usar USER node cuando sea viable.
Dependencias vulnerables	Node.js / React	GitHub Dependabot como SCA.
Patrones inseguros en código	Frontend / Backend	Snyk como SAST.
Vulnerabilidades en imágenes	Docker images	Escaneo con Trivy.
Puertos o servicios expuestos	Staging / preprod	Revisión con Nmap.
Fallos en ejecución	Aplicación levantada	Pruebas DAST con OWASP ZAP.
XSS o entradas maliciosas	Formulario de tareas	Validación y saneamiento de entradas.

2.6. Implementación DevSecOps paso a paso

La implementación DevSecOps se organiza en varias fases secuenciales. El objetivo es que la seguridad forme parte del flujo normal de trabajo, desde el commit hasta el despliegue en preproducción.

Cultura de seguridad

- Revisar las versiones de los servicios utilizados para evitar desplegar contenedores vulnerables.
- Aplicar técnicas de desarrollo seguro, evitando funciones que puedan exponer información sensible.
- Formar a los desarrolladores en buenas prácticas adaptadas al stack tecnológico del proyecto, como Node.js y React.

Control de versiones y ramas

- Rama main: contiene la versión oficial y estable de la aplicación.
- Rama preprod: se utiliza para validar cambios antes de actualizar main.
- Ramas feature/fix: se crean de forma temporal para nuevas funcionalidades o correcciones concretas.

Incorporar seguridad en el desarrollo

- Crear un archivo .env para centralizar credenciales fuera del código fuente.
- Añadir .env al .gitignore para evitar que se suba a GitHub.
- Sustituir credenciales en compose.yaml por variables de entorno, por ejemplo \${MYSQL_ROOT_PASSWORD}.
- Introducir USER node en Dockerfile para evitar ejecución como root cuando sea viable.
- Fijar versiones concretas de servicios, como phpMyAdmin 5.2.3-fpm-alpine.

Automatización CI/CD

- El pipeline se activa cuando un desarrollador realiza un commit hacia la rama preprod.
- Antes de construir imágenes Docker se ejecutan análisis estáticos de código y revisión de dependencias.
- Si se detectan vulnerabilidades altas o críticas, la fusión se bloquea mediante Quality Gates.
- Si las pruebas son satisfactorias, el código queda preparado para actualizar el entorno de preproducción.

Herramientas de seguridad seleccionadas

Se priorizan herramientas integrables con GitHub y adecuadas para una Web-App To-Do basada en Node.js, React, Docker y MySQL. La siguiente tabla resume el uso de cada herramienta, incluyendo ventajas y limitaciones para justificar su incorporación al flujo DevSecOps.

Herramienta	Uso en el proyecto	Ventaja principal	Limitación
Snyk (SAST)	Analizar código y dependencias de la Web-App.	Integración sencilla con GitHub e informes claros.	Puede generar falsos positivos y algunas funciones son avanzadas.
GitHub Dependabot (SCA)	Revisar dependencias y proponer actualizaciones.	Nativo de GitHub y útil para mantener dependencias actualizadas.	Puede generar ruido si no se priorizan alertas.
GitHub Secret Scanning	Detectar tokens, contraseñas o secretos subidos por error.	Previene fugas de credenciales desde el repositorio.	Puede requerir configuración para secretos personalizados.
Trivy	Escanear imágenes Docker, Dockerfile y configuraciones.	Rápido e integrable en CI/CD.	Requiere priorizar CVE para evitar tratar todas las alertas igual.
Nmap	Comprobar puertos y servicios expuestos en staging.	Valida la superficie de exposición real.	No sustituye pruebas de código ni análisis de dependencias.
OWASP ZAP (DAST)	Probar dinámicamente la aplicación levantada.	Detecta fallos visibles en ejecución.	Consume más tiempo y necesita entorno desplegado.

Flujo de pipeline propuesto

Etapas	Control DevSecOps	Herramienta	Resultado esperado
Commit a preprod	Activación del pipeline	GitHub Actions	Se ejecutan las comprobaciones automáticas.
Secret scanning	Búsqueda de credenciales expuestas	GitHub Secret Scanning	Detectar tokens o claves subidas por error.
SAST	Análisis estático de código	Snyk	Detectar patrones inseguros antes de construir la imagen.
SCA	Análisis de dependencias	GitHub Dependabot	Detectar CVE y proponer actualizaciones.
Contenedores	Escaneo de imagen y Dockerfile	Trivy	Detectar vulnerabilidades y malas configuraciones.
Staging	Comprobación de servicios expuestos	Nmap	Validar que solo estén accesibles los puertos necesarios.
DAST	Pruebas dinámicas sobre la aplicación	OWASP ZAP	Detectar fallos en ejecución antes de promocionar a main.
Quality Gate	Bloqueo automático del merge	GitHub Actions / reglas de rama	Impedir la fusión si existen riesgos altos o críticos.
Entrega controlada	Despliegue en preproducción	Docker Compose	Validar antes de la rama main estable.

Concienciación y mejora continua

La mejora continua se apoya en políticas de contraseñas seguras, simulaciones de phishing, sesiones sobre amenazas actuales y OWASP Top 10, junto con la revisión periódica de dependencias, imágenes y configuraciones tras cada cambio relevante.

2.7. Diagrama S-SDLC y DevSecOps

El siguiente diagrama representa el flujo propuesto para incorporar S-SDLC y DevSecOps en la aplicación To-Do. Se mantiene el esquema inicial del grupo para conservar coherencia con el material generado anteriormente.

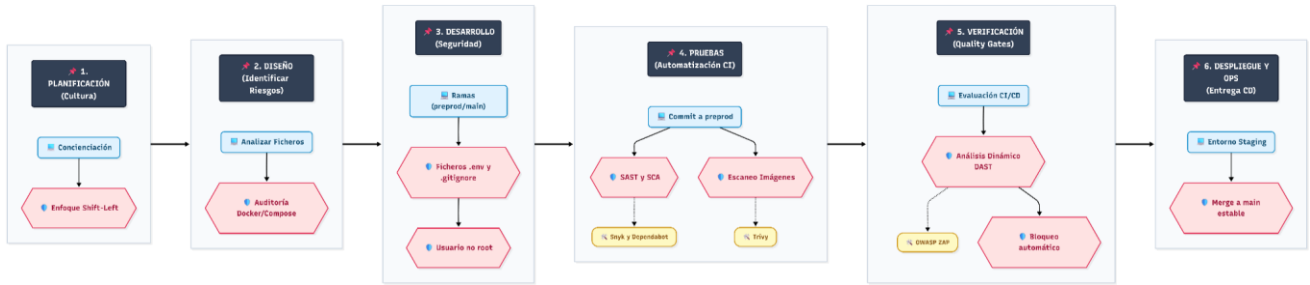


Diagrama S-SDLC y DevSecOps utilizado como base del proyecto.

Fase	Aplicación en el proyecto
Planificación	Cultura de seguridad, enfoque Shift-Left y definición de ramas main, preprod y feature/fix.
Diseño	Identificación de riesgos en compose.yaml, Dockerfile, dependencias y exposición de servicios.
Desarrollo	Uso de .env, .gitignore, usuario no root, versiones controladas y revisión de secretos.
Pruebas	Ejecución de Snyk, GitHub Dependabot, GitHub Secret Scanning, Trivy y Nmap dentro del flujo de validación.
Verificación	OWASP ZAP como DAST y Quality Gates para bloquear vulnerabilidades altas o críticas.
Despliegue	Validación en staging/preprod y promoción controlada a main solo si el pipeline queda en verde.

3. Actividad 3: evaluación de seguridad mediante herramientas automáticas

Esta segunda parte continúa el trabajo anterior y se centra en comprobar la seguridad del proyecto mediante herramientas automáticas. Para cumplir el objetivo, se seleccionan **cinco técnicas** relevantes para la aplicación To-Do y se ejecutan sobre una **versión modificada** de forma controlada para provocar alertas y observar el comportamiento de las herramientas.

Las modificaciones introducidas no tienen como objetivo mejorar la aplicación, sino generar un **escenario inseguro de laboratorio**. De este modo se puede comprobar si las herramientas detectan vulnerabilidades en dependencias, secretos, configuración de contenedores, red y comportamiento dinámico de la aplicación.

3.1. Herramientas seleccionadas y justificación

Las herramientas se eligieron porque cubren distintos puntos del ciclo de vida seguro: análisis estático, análisis de dependencias, detección de secretos, escaneo de red y análisis dinámico de la aplicación en ejecución.

Herramienta	Tipo	Motivo de selección	Pros	Contras
Snyk	SAST	Auditar código fuente y dependencias de backend/frontend en fases tempranas.	Integra Shift-Left, se conecta con GitHub y ayuda a priorizar riesgos.	Puede generar falsos positivos y no ve problemas puramente dinámicos.
GitHub Dependabot	SCA	Detectar dependencias vulnerables u obsoletas y generar alertas o PR.	Integración nativa en GitHub y apoyo a SBOM/cumplimiento.	Puede alertar sobre librerías vulnerables no explotables en la práctica.
GitHub Secret Scanning	Secret scanning	Monitorizar el repositorio para detectar claves, tokens o contraseñas subidas por error.	Ayuda a prevenir fugas de información sensible durante la codificación.	Depende de patrones conocidos; puede no detectar secretos falsos o formatos no reconocidos.
Nmap	Escaneo de red	Comprobar puertos y servicios expuestos por los contenedores Docker.	Rápido, ligero y útil para validar la superficie de exposición.	No analiza lógica interna de la aplicación ni vulnerabilidades de código.
OWASP ZAP	DAST	Evaluar la aplicación levantada simulando pruebas externas.	Detecta problemas de cabeceras, CSP, XSS y errores en ejecución.	Consume más tiempo y puede ser agresivo si se ejecuta en producción sin control.

3.2. Alteraciones introducidas para generar alertas

Para poder observar resultados reales en las herramientas, se alteró intencionadamente la aplicación y algunos ficheros de configuración. Las modificaciones se documentan para que quede claro qué se buscaba detectar y qué efecto se esperaba en la evaluación.

Fichero / componente	Alteración realizada	Objetivo de la modificación	Herramienta esperada
Dockerfile	Uso de node:20.1.0, versión con vulnerabilidades conocidas.	Provocar alertas por imagen/base vulnerable.	Dependabot / Snyk / Trivy
Dockerfile	COPY configurado para copiar todo el contenido.	Aumentar riesgo de exposición de ficheros sensibles.	Snyk / Trivy
Dockerfile	No se especifica USER node.	Ejecutar procesos como root dentro del contenedor.	Snyk / Trivy
compose.yaml	Se añade privileged: true al servicio proxy.	Detectar ejecución con privilegios excesivos.	Trivy / revisión DevSecOps
compose.yaml	Se expone MySQL al host.	Comprobar si el puerto 3306 queda abierto.	Nmap
backend/src/index.js	CORS permisivo, errores verbosos, secretos en código y uso de eval().	Provocar alertas de SAST y DAST.	Snyk / ZAP / Secret Scanning

backend/src/routes/addItem.js	Uso de ...req.body, inyección de comandos y console.log(req.headers).	Detectar exposición de información e inyección de comandos.	Snyk / ZAP
-------------------------------	---	---	------------

En la documentación técnica se evidencian estos cambios mediante capturas de **Dockerfile**, **compose.yaml** y ficheros del backend. También se muestra la referencia de vulnerabilidades asociadas a **Node.js 20.1.0** y los cambios inseguros introducidos en el código.

Nodejs » Node.js » 20.1.0

Vulnerabilities (43) Metasploit Modules

Version names

- Nodejs Node.js 20.1.0
- cpe:2.3:a:nodejs:node.js:20.1.0:*:*:*:*:*
- cpe:/a:nodejs:node.js:20.1.0:~::~~::

Product information

- <https://nodejs.org/en/download/releases> Version
- <https://nodejs.org/en/> Vendor
- https://github.com/nodejs/node/blob/main/doc/changelogs/CHANGELOG_V20.md Change Log

Vulnerabilities by types/categories

Year	Overflow	Memory Corruption	Sql Injection	XSS	Directory Traversal	File Inclusion	CSRF	XXE	SSRF	Open Redirect	Input Validation
2023	0	0	0	0	4	0	0	0	0	0	0
2024	0	0	0	0	3	0	0	0	0	0	0
2025	0	0	0	0	0	0	0	0	0	0	0
2026	0	0	0	0	0	0	0	0	0	0	0
Total					7						

Vulnerabilities by impact types

Year	Code Execution	Bypass	Privilege Escalation	Denial of Service	Information Leak
2023	0	0	2	2	0
2024	1	0	0	2	0
2025	0	0	0	0	0
2026	0	0	0	3	0
Total	1		2	7	

Referencia de vulnerabilidades asociadas a Node.js 20.1.0 utilizada para justificar el cambio de versión.

<pre> services: # Service: proxy # # This service is a reverse proxy that will route requests to the appropriate # service, think of it like a HTTP router or a load balancer, it simply # forwards requests and allows us to simulate the final version of the # application where the frontend and backend are bundled together. We can # also use it to route requests to phpmyAdmin, which won't be accessible at # localhost, but at db.localhost. # # The image for this service comes directly from Docker Hub and is a Docker # Official Image. Since Traefik can be configured in a variety of ways, we # configure it here to watch the Docker events for new containers and to use # their labels for configuration. That's why the Docker socket is mounted. # # We also expose port 80 to connect to the proxy from the host machine. proxy: image: traefik:v3.6 command: --providers.docker ports: - 80:80 volumes: - /var/run/docker.sock:/var/run/docker.sock </pre>	VERSION ORIGINAL	<pre> services: # Service: proxy # # This service is a reverse proxy that will route requests to the appropriate # service, think of it like a HTTP router or a load balancer, it simply # forwards requests and allows us to simulate the final version of the # application where the frontend and backend are bundled together. We can # also use it to route requests to phpmyAdmin, which won't be accessible at # localhost, but at db.localhost. # # The image for this service comes directly from Docker Hub and is a Docker # Official Image. Since Traefik can be configured in a variety of ways, we # configure it here to watch the Docker events for new containers and to use # their labels for configuration. That's why the Docker socket is mounted. # # We also expose port 80 to connect to the proxy from the host machine. proxy: image: traefik:v3.6 command: --providers.docker privileged: true ports: - 80:80 volumes: - /var/run/docker.sock:/var/run/docker.sock </pre>	VERSION INSEGURA
--	-------------------------	---	-------------------------

Modificación en Dockerfile: imagen vulnerable, COPY amplio y ausencia de USER node.

<pre>mysql: image: mysql:9.3 volumes: - todo-mysql-data:/var/lib/mysql environment: MYSQL_ROOT_PASSWORD: secret MYSQL_DATABASE: todos healthcheck: test: ["CMD", "mysqladmin", "ping", "-h", "localhost"] interval: 5s timeout: 5s retries: 5 # Service: phmyadmin</pre>	VERSION ORIGINAL	<pre>mysql: image: mysql:9.3 volumes: - todo-mysql-data:/var/lib/mysql ports: - "0.0.0.0:3306:3306" environment: MYSQL_ROOT_PASSWORD: secret MYSQL_DATABASE: todos healthcheck: test: ["CMD", "mysqladmin", "ping", "-h", "localhost"] interval: 5s timeout: 5s retries: 5</pre>	VERSION INSEGURA
--	---	--	---

Modificación en compose.yaml: privileged:true y exposición de MySQL al host.

<pre>1 const express = require('express'); 2 const app = express(); 3 const db = require('./persistencia'); 4 const getGreeting = require('./routes/getGreeting'); 5 const getItems = require('./routes/getItems'); 6 const addItem = require('./routes/addItem'); 7 const updateItem = require('./routes/updateItem'); 8 const deleteItem = require('./routes/deleteItem'); 9 10 app.use(express.json()); 11 app.use(express.static(__dirname + '/static')); 12 13 app.get('/api/greeting', getGreeting); 14 app.get('/api/items', getItems); 15 app.post('/api/items', addItem); 16 app.put('/api/items/:id', updateItem); 17 app.delete('/api/items/:id', deleteItem); 18 19 db.init() 20 .then(() => { 21 app.listen(3000, () => console.log('Listening on port 3000')); 22 }) 23 .catch(err) => { 24 console.error(err); 25 process.exit(1); 26 }); 27 28 const gracefulShutdown = () => { 29 db.tearDown() 30 .catch(() => {}); 31 process.exit(1); 32 }; 33 34 process.on('SIGINT', gracefulShutdown); 35 process.on('SIGTERM', gracefulShutdown); 36 process.on('SIGUSR2', gracefulShutdown); // Sent by nodemon 37</pre>	VERSION ORIGINAL	<pre>1 const express = require('express'); 2 const app = express(); 3 const db = require('./persistencia'); 4 const getGreeting = require('./routes/getGreeting'); 5 const getItems = require('./routes/getItems'); 6 const addItem = require('./routes/addItem'); 7 const updateItem = require('./routes/updateItem'); 8 const deleteItem = require('./routes/deleteItem'); 9 10 const { exec } = require('child_process'); 11 const cors = require('cors'); 12 app.use(cors({ origin: '*' })); 13 14 app.use(express.json()); 15 app.use(express.static(__dirname + '/static')); 16 17 app.get('/api/greeting', getGreeting); 18 app.get('/api/items', getItems); 19 app.post('/api/items', addItem); 20 app.put('/api/items/:id', updateItem); 21 app.delete('/api/items/:id', deleteItem); 22 23 app.post('/api/network-test', (req, res) => { 24 const url = req.body.url; 25 exec('ping -c 1 ' + url, (error, stdout, stderr) => { 26 if (error) { 27 return res.status(500).send('Error al ejecutar ping: ' + error); 28 } 29 res.send('ping: ' + stdout); 30 }); 31 }); 32 33 app.use((err, req, res, next) => { 34 res.status(500).send('Error interno del servidor (Stack Trace): ' + err.stack); 35 }); 36 37 db.init() 38 .then(() => { 39 app.listen(3000, () => console.log('Listening on port 3000')); 40 }) 41 .catch(err) => { 42 console.error(err); 43 process.exit(1); 44 }); 45 46 const gracefulShutdown = () => { 47 db.tearDown() 48 .catch(() => {}); 49 process.exit(1); 50 }; 51 52 process.on('SIGINT', gracefulShutdown); 53 process.on('SIGTERM', gracefulShutdown); 54 process.on('SIGUSR2', gracefulShutdown); // Sent by nodemon 55</pre>	VERSION INSEGURA
--	---	--	---

Cambios inseguros en backend/src/index.js: CORS permisivo, ejecución insegura y errores verbosos.

<pre>1 const db = require('./persistencia'); 2 const { uuid } = require('uuid'); 3 4 module.exports = async (req, res) => { 5 const item = { 6 id: uuid(), 7 name: req.body.name, 8 completed: false, 9 }; 10 11 await db.storeItem(item); 12 res.send(item); 13 }; 14</pre>	VERSION ORIGINAL	<pre>1 const db = require('./persistencia'); 2 const { uuid } = require('uuid'); 3 4 const { exec } = require('child_process'); 5 6 module.exports = async (req, res) => { 7 8 const item = { 9 id: uuid(), 10 completed: false, 11 ...req.body 12 }; 13 14 exec('echo "Procesando nueva tarea: ' + req.body.name + '"', (err, stdout, stderr) => { 15 if (err) { 16 console.error("Error en la ejecución:", stderr); 17 } 18 }); 19 20 console.log("Headers de la petición no sanitizados:", req.headers); 21 22 await db.storeItem(item); 23 res.send(item); 24 }</pre>	VERSION INSEGURA
---	---	--	---

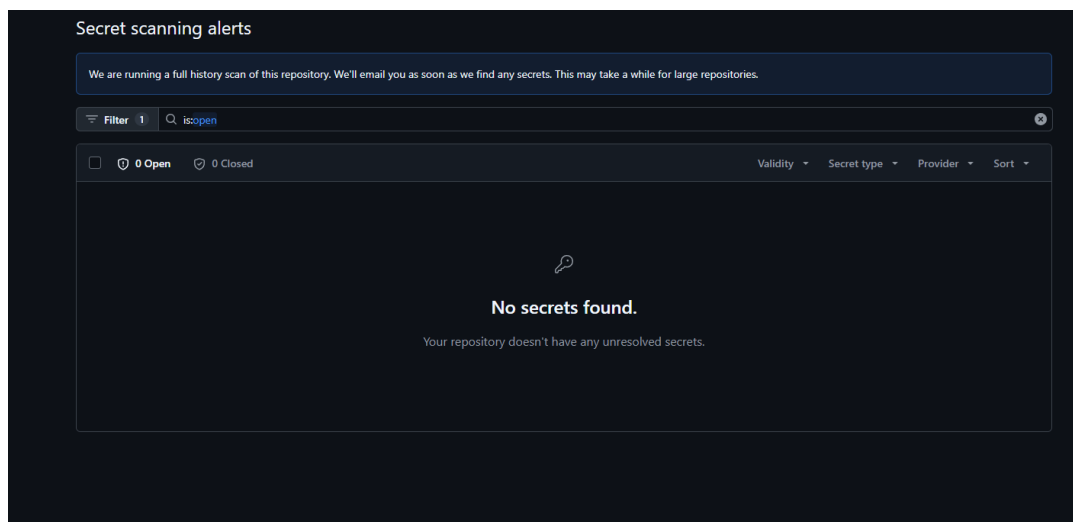
Cambios en rutas del backend: uso inseguro de req.body, ejecución de comandos y logging de cabeceras.

3.3. Resultados de las pruebas automáticas

Tras introducir las modificaciones inseguras, se ejecutaron las herramientas seleccionadas. Los resultados permiten comprobar qué tipo de hallazgos detecta cada herramienta y qué limitaciones aparecen en la práctica.

GitHub Secret Scanning

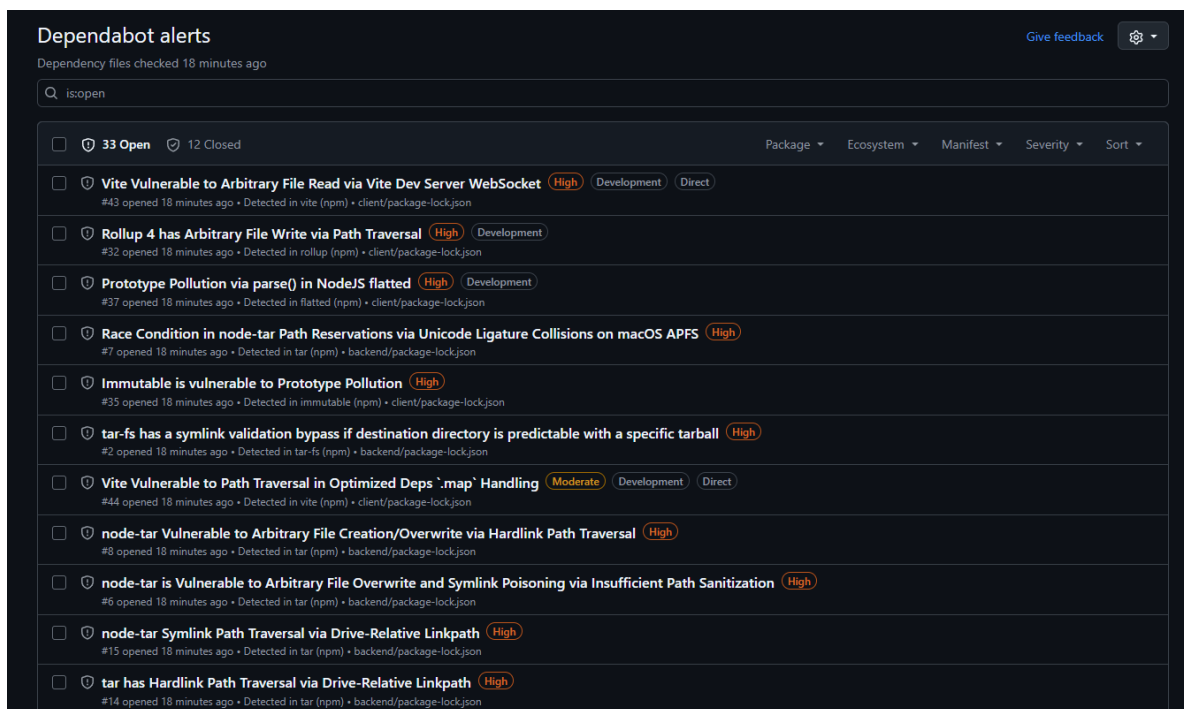
Se inyectaron credenciales falsas en el código para probar la detección automática de secretos. GitHub Secret Scanning no generó alerta en este caso, lo que se interpreta como un **falso negativo**. Esto confirma que la herramienta depende de patrones y proveedores reconocidos, por lo que no sustituye a una revisión segura del código ni a una correcta gestión de `.env` y `.gitignore`.



GitHub Secret Scanning sin alertas detectadas tras introducir credenciales falsas.

GitHub Dependabot - SCA

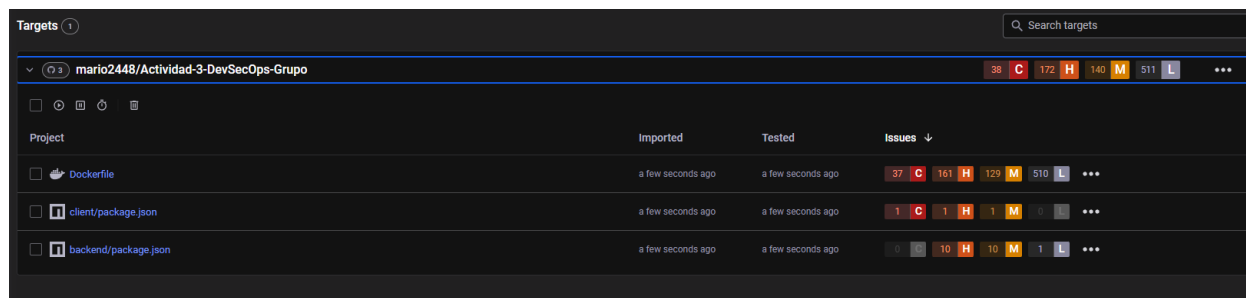
Al bajar intencionadamente la imagen base a node:20.1.0 en el Dockerfile, Dependabot detectó **33 vulnerabilidades** abiertas, incluyendo alertas críticas relacionadas con lectura arbitraria de archivos, **path traversal** y **prototype pollution**. Esto demuestra la utilidad del SCA para vigilar dependencias y versiones con fallos públicos conocidos.



Alertas de GitHub Dependabot generadas por dependencias y versiones vulnerables.

Snyk - SAST

Snyk analizó el código y la configuración del proyecto en estático. El análisis del **Dockerfile** fue el más afectado, detectando vulnerabilidades críticas y altas derivadas de la imagen base vulnerable, la copia completa del directorio y la ausencia de un **usuario no root**. Este resultado muestra la utilidad del SAST para detectar problemas antes del despliegue.



Resultado de Snyk con vulnerabilidades críticas y altas detectadas.

Nmap - escaneo de red

Se escaneó el entorno local para revisar los puertos de los contenedores en ejecución. El resultado confirmó que el puerto **3306/tcp** de MySQL estaba abierto, validando la vulnerabilidad introducida en **compose.yaml** al exponer la base de datos al host.

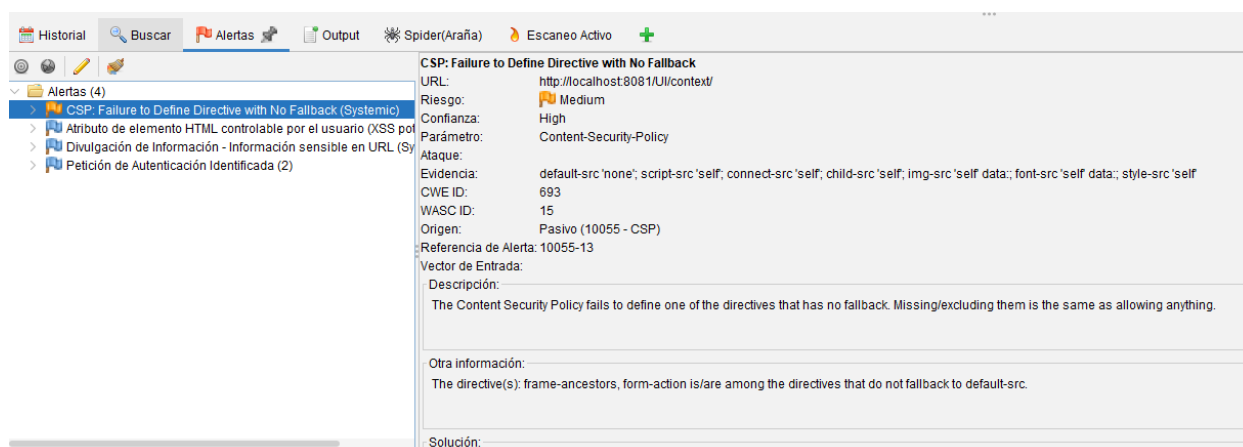
```
C:\Users\mario\Downloads\getting-started-todo-app-insecured\getting-started-todo-app-insecured>nmap localhost
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-05 03:52 +0200
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000070s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 985 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
80/tcp    open  http
135/tcp   open  msrpc
445/tcp   open  microsoft-ds
902/tcp   open  iss-realservice
912/tcp   open  apex-mesh
2008/tcp  open  conf
2179/tcp  open  vmrpd
2869/tcp  open  icslap
3306/tcp  open  mysql
5357/tcp  open  wsddapi
5555/tcp  open  freeciv
8080/tcp  open  http-alt
8080/tcp  open  http-proxy
8090/tcp  open  opsmessaging

Nmap done: 1 IP address (1 host up) scanned in 0.26 seconds
```

Resultado de Nmap mostrando puertos abiertos, incluido MySQL en 3306/tcp.

OWASP ZAP - DAST

OWASP ZAP se ejecutó contra la aplicación levantada para evaluar su comportamiento real. La herramienta detectó alertas en ejecución, destacando problemas de política de seguridad de contenido (**CSP**), atributos HTML controlables por el usuario y posibles vías de exposición de información. Este resultado complementa a las herramientas estáticas, ya que permite observar fallos visibles solo cuando la aplicación está desplegada.



Alertas de OWASP ZAP sobre la aplicación en ejecución.

3.4. Tabla resumen de resultados

Herramienta	Tipo de prueba	Alteración detectada	Resultado	Conclusión
GitHub Secret Scanning	Detección de secretos	Credenciales falsas en código	No generó alerta	Falso negativo útil para evidenciar limitaciones.
GitHub Dependabot	SCA	node:20.1.0 y dependencias vulnerables	33 vulnerabilidades abiertas	Detecta correctamente CVE y librerías vulnerables.
Snyk	SAST	Dockerfile inseguro y patrones vulnerables	Alertas críticas y altas	Eficaz para detectar problemas antes del despliegue.
Nmap	Red	MySQL expuesto al host	Puerto 3306/tcp abierto	Valida superficie de exposición de servicios.
OWASP ZAP	DAST	App vulnerable en ejecución	Alertas por CSP, XSS e información	Complementa SAST/SCA al analizar comportamiento real.

3.5. Relación con S-SDLC y DevSecOps

La actividad demuestra la utilidad de combinar herramientas en distintas fases del ciclo de vida. **Snyk** y **Dependabot** encajan en fases tempranas de desarrollo e integración; **Secret Scanning** protege el repositorio durante la codificación; **Nmap** permite validar la exposición de servicios en staging; y **OWASP ZAP** aporta una visión dinámica antes de promocionar el código a una rama estable.

El uso conjunto de estas herramientas mejora la cobertura del pipeline DevSecOps, ya que ninguna técnica por separado ofrece visibilidad completa. Las pruebas estáticas ayudan a anticiparse, las pruebas de composición identifican dependencias vulnerables, las pruebas de red validan la superficie expuesta y el DAST comprueba el comportamiento real de la aplicación desplegada.

4. Conclusión conjunta

La primera parte del trabajo permitió construir una base **DevSecOps** para la Web-App To-Do, incorporando **S-SDLC**, control de versiones, ramas, automatización CI/CD, herramientas de seguridad y Quality Gates. La segunda parte amplía esa base mediante la ejecución práctica de **herramientas automáticas** de evaluación, generando resultados concretos sobre una versión controladamente vulnerable de la aplicación.

El trabajo demuestra que la seguridad debe tratarse como un proceso continuo. Diseñar la arquitectura, documentar riesgos, configurar Docker de forma segura y automatizar pruebas son pasos complementarios. Además, introducir vulnerabilidades de laboratorio permite comprender mejor cómo actúa cada herramienta y qué limitaciones tiene.

Como conclusión, el proyecto mantiene una Web-App sencilla y comprensible, pero permite evidenciar un flujo de seguridad completo: desde la planificación y diseño hasta la ejecución de pruebas SAST, SCA, Secret Scanning, análisis de red y DAST. Esta combinación ofrece una visión más realista de cómo aplicar desarrollo seguro en un entorno DevSecOps.